ATIVIDADE PAREADA: STRUCTS E ALOCAÇÃO DINÂMICA

1. INSTRUÇÕES DE OPERAÇÃO

- **PILOTO (Digitador):** Responsável por escrever as definições de tipos e a sintaxe de alocação. Deve questionar o uso do operador seta (->) em vez do ponto (.).
- **COPILOTO (Navegador):** Responsável por garantir que nenhum malloc fique sem o seu free correspondente e por revisar a lógica de ponteiros.
- **REGRAS DO CÓDIGO:** Proibido o uso de variáveis globais; obrigatório o uso de protótipos no cabeçalho.

2. DEFINIÇÃO DOS TIPOS (ESTRUTURAS)

A primeira tarefa da dupla é criar o arquivo lista3.h e definir as seguintes estruturas com os nomes exatos de campos abaixo:

1. **struct Ponto**: Representa uma coordenada no espaço.
    - Campos: x (inteiro), y (inteiro).
2. **struct Aluno**: Representa um registro acadêmico.
    - Campos: ra (inteiro), nota (ponto flutuante/float).
3. **struct Circulo**: Representa uma forma geométrica complexa.
    - Campos: raio (ponto flutuante/float), centro (do tipo struct Ponto).

3. CHECKLIST ANTI-VAZAMENTO (MEMORY LEAK)

Como observado nos diagnósticos da turma, a gestão de memória é um ponto crítico. Para validar a atividade, a dupla deve:

1. **Compilar com -g**: gcc main.c lista3.c -o programa -g -Wall -Wextra -lm.
2. **Executar o Valgrind**: valgrind --leak-check=full ./programa.
3. **Confirmar o "Leak Summary"**: O relatório final deve indicar 0 bytes in 0 blocks em uso ao final da execução.

4. ROTEIRO DE 10 EXERCÍCIOS SEQUENCIAIS

Nível 1: Alocação Simples

1. **Criação de Ponto:** Crie uma função que aloque dinamicamente uma struct Ponto e inicialize x e y com valores recebidos por parâmetro.
2. **Impressão com Seta:** Implemente uma função que receba um ponteiro para Ponto e exiba suas coordenadas. **Atenção:** Use obrigatoriamente o operador ->.

Nível 2: Vetores Dinâmicos de Structs

3. **Alocação de Turma:** Crie uma função que receba um inteiro n e aloque um vetor dinâmico de n elementos do tipo struct Aluno.
4. **Processamento de Dados:** Implemente uma função para ler os dados (RA e Nota) de todos os alunos da turma alocada.
5. **Média da Turma:** Crie uma função que receba o ponteiro da turma e o tamanho, retornando a média das notas.

Nível 3: Manipulação e Realloca

6. **Busca de Aluno:** Crie uma função que busque um aluno pelo RA e retorne o ponteiro para a posição exata dele na memória (ou `NULL` se não encontrar).
7. **Expansão de Memória:** Use a função `realloc` para aumentar o tamanho do vetor da turma em 1 posição, simulando a matrícula de um novo aluno.
8. **Cálculo com Composição:** Crie uma função que aloque dinamicamente uma `struct Circulo` e calcule sua área (Área = pi* raio^2).

Nível 4: Integridade e Clonagem

9. **Clonagem de Dados (Deep Copy):** Implemente uma função que receba a turma original e crie uma cópia exata em um novo endereço de memória alocado dinamicamente.
10. **Liberação Segura: Crie uma função para liberar a memória de qualquer ponteiro e, imediatamente após o free, atribua NULL ao ponteiro para evitar ponteiros soltos.**

Atividade adicional: crie 1 main para invocar essas funções e garantir a liberação de memória.